

实操手册：产品经理的Skills如何迭代？

原创 糖糖 糖糖算力营 2026年4月22日 19:30 北京

把上一篇没说完的话补上

为什么产品经理skill要常迭代，因为它本质上是组织能力

那到底怎么迭代？

大多数人走到这里，最容易犯的错误就是把“迭代 skill”理解成“再改一版 prompt”。

开头多加几句背景，结尾补几条要求，换几个更高级的词，看起来像升级了，实际上常常没解决什么问题。

真正需要迭代的，往往不是措辞，而是这套 skill 背后那一整条工作链路：

- 它到底需要什么输入
- 依赖什么判断
- 在哪些节点最容易失真
- 输出给谁，谁来审阅
- 出了问题之后怎样回灌。

skill 不是靠灵感和文学创作优化的，而是靠复盘数据、组织约束和审阅闭环迭代出来的。



先判断：到底该改 skill 的哪一层

产品经理的 skill 干的也就那点破事儿，如果失效，通常有四种原因：

第一种，**触发条件错了**。这个 skill 本来就不该在这个阶段被调用：

- 问题还没定义清楚，就直接进入需求拆解；
- 用户研究证据还很薄，就开始让 AI 输出路线图。

第二种，**输入结构不够**。给 AI 的上下文只有一句目标、两段抱怨和一份旧文档，缺角色、缺场景、缺指标、缺历史约束。

第三种，**判断规则没显性化**。很多 skill 看起来流程完整，但把最关键的判断留给了“默认常识”。比如：

- 优先级到底按收入、客户影响、战略窗口还是实施复杂度来权衡；
- 需求拆分到底以业务闭环、系统边界还是版本节奏为单位。

第四种，**交付接口不对**。输出本身也许不差，但并不是开发、测试、UX 真正拿来工作的格式。

可执行的迭代方法

试试固定成下面这 5 步。

第一步：收集“失败样本”

skill 真正该改的地方，都藏在失败样本里，这也是 AI 最喜欢的 few-shot。失败样本拆解成 5 类输入：

1. **原始任务** 当时希望 AI 帮你完成什么。
2. **原始输入** 你喂给 AI 的文档、背景、约束、提示词。
3. **原始输出** AI 实际产出了什么。
4. **真实结果** 这份输出最后为什么没进入团队流程，或者进入后为什么出了偏差。
5. **人工修正** 最后是怎么被评审、开发、UX 或老板改掉的。

这一步不要急着让 AI “重写更好的版本”，先让它做诊断。

你现在不是内容生成器，而是 skill 复盘分析师。

我会给你一轮真实项目中的 skill 调用记录，请你判断这次失败主要出在哪一层：

1. 触发时机错误
2. 输入信息缺失
3. 判断规则缺失
4. 输出接口不匹配
5. 多个问题叠加

请按以下格式输出：

1. 问题归因
2. 缺失信息清单
3. 本应提前显性化的判断规则
4. 建议新增或修改的输入字段
5. 建议新增或修改的审阅节点

不要直接重写 skill，先做诊断。

这一步的目标，不是得到答案，而是把“为什么这次跑偏”拆清楚。

第二步：补的不是感受，而是结构化上下文

我个人非常讨厌在复盘会上听到“**我觉得这里不太好**”，没人在乎你的感受。

开口之前，先把这句话改成：“**我有理由认为这里不太好，因为……**”，大家在意的，是你的事实和逻辑。

基于这个陈述，你就可以让AI给你整理类似这样的信息：

适用场景 这个 skill 适用于哪类项目、哪种阶段、哪种决策。

必填输入 没有哪些信息，AI 就不允许往下跑。

关键约束 必须优先考虑哪些业务目标、组织边界和工程现实。

输出对象 这份结果最终是给谁用的，开发、UX，还是客户成功。

完成标准 什么样的输出才算能进入下一步，不是“写得完整”，而是“别人接得住”。

这些信息一旦清楚，AI 改写质量会明显提高。因为它终于拿到的不是零散意见，而是组织信息的上下文。

第三步：再让 AI 重写的，不只是 prompt，而是 skill 规格

等前两步做完，才进入真正的改写阶段。

这里我建议不要让 AI 直接“优化原文”，而是让它先输出一份 **skill 规格说明**，再根据规格生成新版本。这个规格，也要保存并持续迭代。

提示词模版供：

你现在是 PM skill 架构师。

请基于我提供的 skill 迭代卡、历史失败案例和原始 skill，输出一份新的 skill 规格说明。

规格说明必须包含：

1. skill 目标
2. 适用场景与禁用场景
3. 触发前提

4. 必填输入字段
5. 关键判断规则
6. 输出对象与输出格式
7. 审阅与升级机制
8. 常见失败模式与防错提醒

输出要求：

- 不要先写完整 skill
- 先写规格说明
- 每条规则都要说明“为什么需要它”

等规格说明出来后，开始草拟新版的skill：

请基于以上规格说明，重写 skill。

要求：

- 保留能复用的稳定部分
- 删除本轮已证明无效的规则
- 新增输入检查与失败兜底提醒
- 明确何时停止生成并要求人工补充信息
- 输出内容要能被团队直接拿去执行

第四步：从“审阅文案”到“评估团队公约”

很多产品经理会自己看一遍、觉得更顺了，就算迭代完成。

但这类自我校验通常不够，因为真正决定 skills 是否可用的，往往不是作者本人，而是下一棒。

所以需要一轮轻量评审，重点不是文字，而看接口。

- 产品经理自己审一遍，确认目标、场景、输入要求是否清楚。
- 开发和测试工程师审一遍，确认输出是否足够落地，是否把工程约束写明。
- UX审一遍，确认这个结果是否真的能支持他们进入设计环节。

审阅时可以关注3个问题：

1. 这个 skill 在什么情况下不该被触发？
2. 输出结果能不能直接进入下一环节？
3. 出了问题，团队能不能知道该补什么，而不是重来一遍？

第五步：把修订意见回灌成下一轮组织记忆

最后一步最像“组织能力”，也是最容易漏掉的一步。

每次迭代之后，不要只留下一个新文件。你至少还要留下两样东西：

一份版本说明 这一版到底改了什么，是为了解什么问题。

一份失败模式库 哪些场景容易误触发，哪些输入缺失最常见，哪些规则最容易被误解。

有了这两样东西，下一次再有人接手这个 skill，他看到的就不只是结果，还有这套结果是怎么被逼出来的。



最后三点

- 1. 当skill 从“个人外挂”变成“组织资产”，它就跑的更稳了。因为它沉淀的已经不只是写法，而是团队在真实项目里反复校正过的判断逻辑、协作接口和工作纪律。
- 2. 在没有AI的时代，这样复盘和优化迭代基本是不可能的：不是说真人做不了这些事情，而是很少有团队愿意投入如此巨大的管理成本来做这件事。
- 3. 条件允许，让不同的大模型（Claude、Codex、Gemini、Qoder等），再有一个做最终总结归纳的，甚至可以考虑用CrewAI搭一个简单的多agent研讨。

产品经理AI转型策略 · 目录 ≡

<| 上一篇

抛开事实不谈，这份 AI 讣告写得真漂亮

下一篇 >

为什么你的 AI 只能写总结，别的产品经理已经用AI在挖需求机会了？附skill模板和调试...